

Interprocedural Constant Propagation: A Study of Jump Function Implementations

Dan Grove

Sun Microsystems
2550 Garcia Avenue
Mountain View, California 94043

Linda Torczon

Department of Computer Science
Rice University
Houston, Texas 77251-1892

Abstract

An implementation of interprocedural constant propagation must model the transmission of values through each procedure. In the framework proposed by Callahan, Cooper, Kennedy, and Torczon in 1986, this intraprocedural propagation is modeled with a *jump function*. While Callahan *et al.* propose several kinds of jump functions, they give no data to help choose between them. This paper reports on a comparative study of jump function implementations. It shows that different jump functions produce different numbers of useful constants; it suggests a particular function, called the *pass-through parameter jump function*, as the most cost-effective in practice.

1 Introduction

Procedure calls have long been recognized as an impediment to performance in compiled code. In part, this happens because procedure calls hide information. Information about the environment that a procedure inherits and about the side effects of procedures that it invokes is unavailable to the optimizer. Interprocedural data-flow analysis attempts to overcome this problem by propagating information about data usage and creation among the procedures in a program. The compiler can then take advantage of contextual information to generate better code for

any single procedure. In particular, a compiler can generate better code for a procedure if it knows at compile time which parameters and global variables will have constant values, and what those values will be, when the procedure is invoked. These constants are called *interprocedural constants*.

Interprocedural constants have been used to improve the results of analysis or optimization in a variety of different situations, including computation of dependence information [14], automatic parallelization of sequential routines [9, 15], and procedure cloning [6, 13]. They also improve many scalar optimizations, including intraprocedural constant propagation, dead code elimination, and certain loop transformations.

Shen, Li, and Yew examined the improvement in dependence information that is obtained when the values of interprocedural constants are known [14]. In their experiment, they performed the interprocedural constant propagation manually and added assertions to the program text about constant variable values. In their examination of FORTRAN library routines, they found that knowledge of interprocedural constants significantly decreased the number of nonlinear subscript references. They found that approximately 50 percent of the subscripts which had previously been considered nonlinear were found to be linear in the presence of interprocedural constant information. Because many dependence analyzers are incapable of analyzing nonlinear subscripts, this is an important application of interprocedural constants.

Another arena in which interprocedural constants are valuable is automatic detection of parallelism. Eigenmann and Blume suggested that interprocedural constants are often used as loop bounds [9]. Loop bounds are important for two reasons. First, they improve dependence information, as described above. Second, knowing their values allows the compiler to make informed decisions about the profitability of parallel execution – the number of iterations executed

This research was supported by DARPA through ONR Grant N00014-91-J-1989.

E-mail addresses: Dan Grove: dgrove@eng.sun.com; Linda Torczon: torczon@rice.edu

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

ACM-SIGPLAN-PLDI-6/93/Albuquerque, N.M.

© 1993 ACM 0-89791-598-4/93/0006/0090...\$1.50

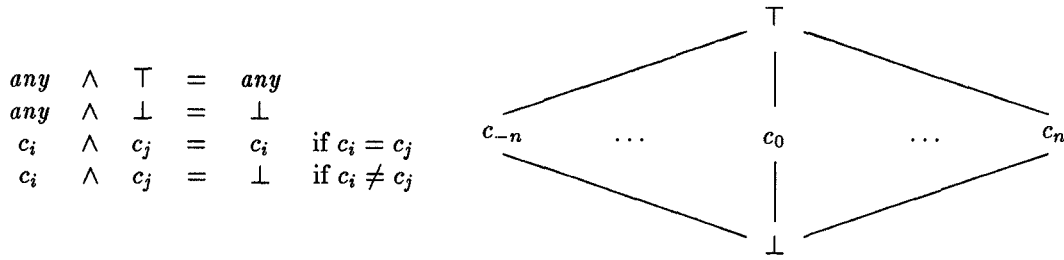


Figure 1: The constant propagation lattice.

by a particular loop is an important factor in determining both the amount of work it represents and the number of processors that it can profitably employ.

Several techniques for discovering interprocedural constants have been proposed [3, 4, 13, 16] (See Section 5, “Other Work”). The study presented in this paper is based on the work of Callahan *et al.* [4]. They suggest several jump functions for their framework. Those jump functions vary in complexity and in the class of interprocedural constants that they are expected to propagate. This study examines the tradeoff between the complexity of the jump functions used and the number of integer constants detected when interprocedural constant propagation is performed on scientific FORTRAN codes. Empirical results are presented.

The remainder of this paper is divided into five sections. Section 2 outlines the framework employed for interprocedural constant propagation. Section 3 describes the jump functions examined. Section 4 discusses both the results of the study and the implementation used. Section 5 sketches related research. Section 6 presents our conclusions.

2 Interprocedural Constant Propagation

The goal of interprocedural constant propagation is to discover, for each procedure p in the program, the set of constants that always hold on entry to p . In data-flow terms, we want to annotate p with a set $\text{CONSTANTS}(p)$ containing $\langle \text{name}, \text{value} \rangle$ pairs. A pair $\langle x, v \rangle \in \text{CONSTANTS}(p)$ indicates that x always has value v when p is invoked. This set conservatively approximates reality; each pair in $\text{CONSTANTS}(p)$ denotes a run-time constant, but not all run-time constants can be found (for example, values read from a file may be combined to form a constant that propagates through the program).

For the purpose of computing $\text{CONSTANTS}(p)$, we formulate the problem in a lattice theoretic framework. With each entry point p , we associate a function that maps formal parameters¹ of p into elements of the *constant propagation lattice* L . Each element of L is one of three types: the lattice *top* element ($\top$), a constant value c , or the lattice *bottom* element ($\perp$). The lattice’s structure is shown on the right of Figure 1. It is defined by the rules for the lattice *meet* operation ($\wedge$). These rules are shown on the left of Figure 1. Here, “*any*” denotes a valid element of L . The lattice, although infinite, has bounded depth. In particular, the value associated with some formal parameter x can be lowered at most twice.

For any formal parameter x , let $\text{VAL}(x)$ represent the best current approximation to the value of x on entry to the procedure. After the analysis is complete, if $\text{VAL}(x) = \perp$, the analysis was not able to determine that x is constant. (x may or may not be constant. If x is constant, the analyzer was unable to discover it.) If $\text{VAL}(x) = c$, x has the constant value c . The value $\top$ is used as an initial approximation for all parameters; x retains the value $\top$ only if the procedure containing x is never called. Given $\text{VAL}(x)$ for each of p ’s formal parameters, $\text{CONSTANTS}(p)$ is simply the set of formal parameters for which VAL is equal to some constant value.

Interprocedural constant propagation is performed on a call graph, G . Each node in G represents a procedure p and each edge $\langle p, q \rangle$ represents a call from p to q . Interprocedural constant propagation has two principal parts: (1) modeling the transmission of values through each procedure and (2) propagating information in VAL sets around the call graph.

¹To simplify the presentation, we extend the definition of a parameter to include global variables. Recall that *formal* parameter refers to the name used inside the called procedure. The reference in the parameter list at the call site is an *actual* parameter.

To model the flow of constant values through a procedure p , we associate with each call site s a set of *forward jump functions*, J_s^y . For an actual parameter y , J_s^y gives the value of y at s as a function of p 's formal parameters, where s is a call site in p . The *support* of J_s^y is the exact set of p 's formal parameters whose values on entry to p are used in the computation of J_s^y . We denote the *support* set of J_s^y as $\text{support}(J_s^y)$.

Procedures can also return constant values. To account for this effect, we must model the transmission of constant values from within a procedure back to the call site that invoked it. With each procedure, we associate a set of *return jump functions*. For each formal parameter x that may be modified by p , a return jump function R_p^x is created to compute the best approximation to x 's value on return from p . The *support* of the function R_p^x is the exact set of formal parameters of p whose values on entry to p are used in the computation of R_p^x . As before, we denote the *support* set of R_p^x as $\text{support}(R_p^x)$.

Given the call graph G and the forward and return jump functions for each procedure, we can propagate the *VAL* sets around the graph using a standard iterative technique [11, 12]. The results presented in this paper were computed using a simple worklist iterative scheme. Alternative formulations based on the binding multi-graph are possible [7]. The method presented by Callahan *et al.* essentially models the binding graph computation on the call graph [4].

Callahan *et al.* showed a single simple example to demonstrate that different jump function techniques produced different results. They fail, however, to provide any experimental evidence that suggests where the tradeoff lies between jump function complexity and the precision of the *CONSTANTS* sets. This paper examines the effectiveness of several jump functions. From this comparative data, we can assess the value of spending additional compile time in jump function evaluation.

3 Jump Functions

To study the tradeoffs between compile time and precision that arise in an implementation, we built an analyzer that can use one of several different jump functions. Our choice of implementation was driven by the goals set for our study: understanding the tradeoff between more complex jump functions and the number of interprocedural constants discovered. Thus, we chose an implementation strategy that differs from the one described by Callahan *et al.* We built a set of jump functions on top of an existing framework for global (*intraprocedural*) value number-

ing [1]. Callahan *et al.* envisioned constructing jump functions before *any* interprocedural data-flow analysis is performed. We found it convenient to simulate the various jump functions on top of global value numbering *after* the computation of interprocedural summary sets (MOD and REF). Section 3.1 describes the four forward jump functions that we considered and the costs associated with each jump function. Section 3.2 describes the single return jump function that we implemented.

3.1 Forward Jump Functions

The four forward jump functions vary both in the complexity of their construction and in the class of interprocedural constants that they can detect. In increasing order of complexity, they are: the literal constant jump function, the intraprocedural constant jump function, the pass-through parameter jump function, and the polynomial parameter jump function. Note that the set of constants propagated by one of these forward jump functions is a subset of the constants propagated by the more complex models that follow it.

Each of the forward jump functions selected for this study can be constructed as initial information before the interprocedural propagation phase begins. It is not necessary to reconstruct the jump functions on each iteration over G . For simplicity in defining these forward jump functions, let $\text{gcp}(y, s)$ represent a function, computed prior to the interprocedural constant propagation phase, that returns the constant value of parameter y at call site s when that value can be determined with intraprocedural constant propagation or value numbering coupled with interprocedural MOD information.

3.1.1 The Literal Constant Jump Function

The literal constant jump function is defined as the following:

$$J_s^y = \begin{cases} c & \text{if } y \text{ is the literal constant } c \text{ at call site } s \\ \perp & \text{otherwise} \end{cases}$$

Because it does not take into account the values of incoming formal parameters, this forward jump function misses opportunities to propagate interprocedural constants through the procedure body. As a result, interprocedural constants can only be propagated over a single edge in G . The only interprocedural constants propagated by this method are those that appear as a literal constant at a call site. (Therefore, this jump function misses any constant globals which are passed implicitly at the call site.)

3.1.2 The Intraprocedural Constant Jump Function

The intraprocedural constant jump function is defined in the following manner:

$$J_s^y = \begin{cases} c & \text{if } gcp(y, s) = c \\ \perp & \text{otherwise} \end{cases}$$

Like the literal constant jump function, this forward jump function ignores any values known on entry to the procedure for incoming formal parameters. Thus, it will only propagate an interprocedural constant along a single edge in G . Because it uses intraprocedural constant propagation to produce the constants propagated at the call sites, it can potentially detect more interprocedural constants than the literal constant jump function. It can also detect constant-valued global variables.

3.1.3 The Pass-through Parameter Jump Function

The pass-through parameter jump function is defined as the following:

$$J_s^y = \begin{cases} c & \text{if } gcp(y, s) = c \\ x & \text{if } y = x \text{ at } s \text{ and } x \text{ is a formal parameter of the calling procedure} \\ \perp & \text{otherwise} \end{cases}$$

This jump function detects situations where a formal parameter is passed unmodified through the procedure body to a call site where it is used as an actual parameter. Because this forward jump function allows interprocedural constants to be propagated through a procedure body, it can propagate interprocedural constants along paths of length greater than one in G .

3.1.4 The Polynomial Parameter Jump Function

The polynomial parameter jump function is defined in the following manner:

$$J_s^y = \begin{cases} c & \text{if } gcp(y, s) = c \\ f(\text{support}(J_s^y)) & \text{if } y \text{ can be represented as a polynomial function } f \text{ of } \text{support}(J_s^y) \\ \perp & \text{otherwise} \end{cases}$$

This jump function takes a more comprehensive approach towards propagating interprocedural constants through a procedure body than the pass-through parameter jump function. Actual parameters are represented as polynomial functions of the incoming values of the formal parameters of p when such a polynomial function can be constructed. All standard integer operations are supported by the routines which generate and evaluate these jump functions.

3.1.5 Costs of Forward Jump Functions

The four forward jump functions vary both in the cost of construction and the time required for solving the interprocedural problem. The literal constant jump function is constructed with a minimal amount of effort – a textual scan of the call sites provides all the required information. Building the other three forward jump functions involves some intraprocedural analysis, so they are more expensive to construct. Our implementation used an SSA-based value numbering scheme to generate these jump functions. Building the SSA graph for a procedure typically requires $O(N)$ steps, where N is the size of the procedure (the maximum of the number of nodes, edges, variable assignments, or variable references in the procedure) [8, page 487]. While the time required for construction of these three jump functions is similar, the complexity of the data structures used by the polynomial parameter jump function is significantly greater than for either the intraprocedural constant or pass-through parameter jump function.

The time required for the interprocedural propagation step also varies among these jump functions. The analysis has three distinct cases.

1. The literal constant and intraprocedural constant jump functions can only propagate constants along paths of length one in the call graph. Thus, the interprocedural propagation can be solved in $O(\sum_s \sum_y \text{cost}(J_s^y))$ time, where s ranges over all call sites, y ranges over all parameters at each call site, and $\text{cost}(J_s^y)$ is the cost of evaluating J_s^y . For the intraprocedural constant jump function, y includes both actual parameters and global variables.
2. The pass-through parameter jump function can pass constants along arbitrary paths in the call graph. Because the constant lattice is shallow, each value can be lowered at most twice – from $\top$ to a constant to $\perp$. When a formal parameter's value is lowered, each actual parameter that uses its value must be re-evaluated. However, each actual parameter depends on exactly one formal parameter. Taken together, these facts suggest that the interprocedural propagation can be completed in $O(\sum_s \sum_y \text{cost}(J_s^y))$ time. Callahan *et al.* present an algorithm that achieves that time bound [4]. The implementation used in our experiment was less efficient. Note that $\text{cost}(J_s^y)$ is greater for the pass-through jump function than for the literal constant and intraprocedural constant jump functions.

3. The polynomial parameter jump function also passes constants along arbitrary paths in the call graph. Each element in $\text{support}(J_s^y)$ can be lowered at most twice, resulting in the re-evaluation of J_s^y . Thus the time required for interprocedural propagation is [4]:

$$O\left(\sum_s \sum_y \text{cost}(J_s^y) \cdot |\text{support}(J_s^y)|\right)$$

In practice, we found that the number of complex polynomial jump functions actually constructed is small. Taken over the program, $\text{cost}(J_s^y)$ approaches the cost of pass-through parameter jump functions and $|\text{support}(J_s^y)|$ approaches 1. These effects allow the solution time for the interprocedural constant propagation to approach that of the simpler forward jump functions, $O(\sum_s \sum_y \text{cost}(J_s^y))$.

3.2 The Return Jump Function

To understand the value of return jump functions, we implemented a single type of return jump function. It is similar to the forward polynomial jump function. The values computed for return jump functions are calculated during an initial bottom-up pass through the call graph. Interprocedural MOD information, return jump functions from routines already analyzed, and intraprocedural constants are used in the construction of each return jump function polynomial.

For simplicity in defining this return jump function, let $\text{rgcp}(x, p)$ represent a function, computed prior to the interprocedural constant propagation phase, that returns the constant value of formal parameter x after invocation of p when that value can be determined with intraprocedural constant propagation coupled with interprocedural MOD information and information provided by return jump functions that have already been computed for call sites within p . Formally, the return jump function is defined as follows:

$$R_p^x = \begin{cases} c & \text{if } \text{rgcp}(x, p) = c \\ g(\text{support}(R_p^x)) & \text{if } x \text{ can be represented} \\ & \text{as a polynomial function } g \text{ of } \text{support}(R_p^x) \\ \perp & \text{otherwise} \end{cases}$$

Because an SSA-based value numbering scheme is used to generate return jump functions, the amount of time required to construct them is typically $O(N)$ – the same as that needed for the polynomial parameter

forward jump function. The data structures needed in this stage are also identical to those required by the polynomial parameter jump function. Each return jump function is evaluated exactly twice at each call site. It is first evaluated during the generation of return jump functions for the calling procedure. This is done in order to expose as many return jump functions as possible in the calling procedure. The second evaluation of the return jump function takes place during the generation of forward jump functions. During this phase, any return jump function that cannot be evaluated as constant using intraprocedural information coupled with other return jump function values is set to $\perp$. The effect of this limitation is that return jump functions that depend on parameters to the calling procedure can never be evaluated as constant.

4 Study

To study the relative effectiveness of the jump functions described in Section 3, we built an analyzer that implements each of them and used it to analyze a collection of FORTRAN programs. This allows a fair comparison of the set of constants discovered with each jump function. We limited the implementation in two ways.

1. The implementation only propagates integer constants. There are two principal reasons for this. First, we were most interested in discovering constants that affect control flow. These include loop bounds, loop strides, and conditions that control `if-then-else` statements. Second, compile-time evaluation of floating-point numbers is problematic, particularly so in an environment like ours where the compiler and application may run on different architectures.
2. The analyzer does not try to track constants in and out of arrays. Dependence analysis might allow the analyzer to track some constant-valued array elements. We elected not to attempt this because the information from interprocedural constant propagation is used as input to the dependence analyzer. Thus, using dependence information inside a jump function would either introduce a phase-ordering problem or require iterative application of dependence analysis and interprocedural constant propagation.² Any references to array elements are initialized to $\perp$.

²We believe that tracking the flow of values in and out of FORTRAN arrays would yield few additional interprocedural constants.

These limitations are in line with the goals of our study – to compare the sets of integer constants discovered with the various jump functions described in Section 3.

4.1 Implementation

The interprocedural constant propagation system used in this study was built on top of an SSA-based value number graph [8] and the call graph abstraction from ParaScope, a set of tools designed to aid in interactive parallelization of FORTRAN programs [5]. Its execution proceeds in four stages: generation of return jump functions, generation of forward jump functions, interprocedural propagation of constants, and recording the results.

Generating return jump functions. In the first phase, a return jump function for each formal parameter modified by a procedure is calculated in a bottom-up walk over the call graph. At each node in the call graph, the analyzer builds an SSA graph for the corresponding procedure [8]. To determine which parameters can have constant values on return from the routine, we value number the SSA graph. The value numbering pass incorporates information provided by already-computed return jump functions for subroutines invoked by the current routine. After the return jump functions have been built, the SSA graph and the value number graph are discarded to conserve memory. The data structures which hold the expressions representing the return jump functions are stored for later use.

Generating forward jump functions. In the second phase, forward jump functions are generated for every call site. In a top-down pass over the call graph, the analyzer builds the SSA graph and the value number graph for each routine. This analysis uses the return jump function expressions built in the first stage. Each parameter associated with a call site receives a value number. The value numbering proceeds in the standard way; it can build an arbitrarily complex representation for an arithmetic expression. The analyzer then constructs an expression tree to represent the parameter’s jump function from the expression produced by value numbering.³ The resulting expression tree is converted into a context-independent representation and stored in the list of jump functions for the parameters of this call site.

Our implementation was intended for experimentation. Of course, simpler representations are possible

for the literal constant, intraprocedural constant, and pass-through parameter jump functions. For example, the pass-through parameter jump functions can be easily derived from either an SSA graph or DEF-USE chains. They can be represented with a simple map from actuals to formals and globals at each call site. A production implementation should consider such simpler representations.

Interprocedural propagation of constants. Once forward jump functions have been built, the compiler must perform interprocedural propagation. Rather than build a specialized solver for this problem, we used ParaScope’s interprocedural data-flow solver. Meet and transfer functions were written for this solver; they evaluate the jump functions. The solver uses an iterative method. Even with this less efficient solver, the problems converged quickly. In our implementation, the cost of intraprocedural analysis dominates the cost of the interprocedural phase.

The meet function is called, in turn, for every parameter along each edge entering a procedure. Meet combines the values (produced by jump functions) flowing from each call site with the current values of the parameters in the procedure. Meet is defined as shown in Figure 1; any non-constant jump function evaluates to $\perp$.

Once the meet has been performed, the values of constant parameters must be transmitted to call sites within the node. If an incoming parameter which was previously evaluated as non-constant is now found to be constant, all non-constant parameters at each call site within that procedure are re-evaluated. This process uses a symbolic expression evaluator to interpret the expression tree that represents the jump function. It uses all available information about incoming parameter values.

If this process finds that a previously non-constant actual parameter is now constant, the iterative data-flow analyzer makes another pass over the call graph. This process continues until no new constants are found during an entire iteration. At this point, the CONSTANTS sets for the nodes are stable, and the data-flow solver halts.

Recording the results. The CONSTANTS sets are written to a single file. Optionally, the analyzer can produce a transformed version of the original source in which the interprocedural constants are textually substituted into the code. The numbers reported in the next section count the number of constants that this option substituted into each program. Metzger and Stroud suggest that this is the right measurement of effectiveness [13]. In their experience, FORTRAN

³The user specifies which forward jump function is desired. The appropriate function is constructed from the information produced by value numbering.

Program	Lines	Procs	Median Lines per Proc	Mean Lines per Proc
adm	3809	98	29	39
doduc	4514	41	78	110
fpppp	1835	38	5	48
linpackd	392	11	25	36
matrix300	153	6	23	26
mdg	856	18	25	48
ocean	1728	37	22	46
qcd	1664	36	32	46
simple	805	8	49	101
snasa7	696	17	34	41
spec77	2904	65	31	45
trfd	401	8	40	50

Table 1: Characteristics of program test suite.

procedures often have constant-valued global variables that are known but irrelevant – that is, they are not referenced inside the procedure. This measure relates more directly to code improvement; it also factors out procedure length and modularity.

4.2 Results

To evaluate the relative effectiveness of the jump functions, we chose a set of scientific FORTRAN programs. We elected to evaluate the SPEC and PERFECT benchmark suites. Because ParaScope does not currently support multiple entry points, some of the PERFECT suite had to be removed from consideration.⁴ Some characteristics of the programs which were tested are shown in Table 1. In general, the programs are of small to medium size, with a fairly high degree of modularity. The line counts exclude comments and blank lines. With the exceptions of **fpppp** and **simple**, a fairly even distribution of code throughout the procedures in each program was found (this is shown by the closeness of the mean lines per procedure and the median lines per procedure in each program). In these two programs, the distribution of the lines per procedure was skewed – a single routine made up a large part of the code in **fpppp** and **simple**. While these programs may not be representative of all styles of scientific computing, the two suites of programs are well known and widely used in studies like this one.

Table 2 shows the number of constants discovered and actually substituted into the code by the ana-

lyzer using each of the jump functions. The first four columns show the numbers of constants obtained by using both forward and return jump functions. The final two columns show the results for polynomial and pass-through jump functions without a return jump function. In our test suite, the polynomial and pass-through parameter techniques found the same set of constants. Both outperformed the intraprocedural jump function. The intraprocedural jump function performed better than the literal jump function.

Return jump functions made no noticeable difference in ten of the thirteen programs. In **doduc** and **mdg**, return jump functions let the analyzer find a few more constants. In **ocean**, the return jump functions more than tripled the number of constants that the analyzer found. By recognizing that the initialization routine at the start of **ocean** resulted in the assignment of constant values to many variables, the analyzer was able to propagate additional constants to routines throughout the program.

Table 3 shows several different techniques. The first column shows results from a propagation using polynomial parameter forward and return jump functions and no MOD information. The second column of numbers is identical with the first column in Table 2. The third column shows the result of a “complete” propagation. To obtain these numbers, the analyzer was run with polynomial parameter forward and return jump functions, using MOD information. After each run, dead code elimination was performed. If any dead code was found, the propagation was performed again from scratch – all of the values in CONSTANTS sets were reset to T.⁵ The final column shows the results of performing a purely intraprocedural constant propagation on each procedure in the program. No constants were propagated between procedures, but interprocedural MOD information was used during the intraprocedural propagation.

To assess the importance of MOD information, we eliminated it from both the initial information and the jump functions. Comparing the results (see column 1) against those obtained with MOD information (see column 2), we see a substantial difference. In any program where constants were found, using MOD information exposed additional constants. The numbers are particularly striking in **adm**, **linpackd**, **matrix300**, **ocean**, **simple**, and **spec77**. Without MOD information, the value numbering pass (see Section 4.1) had to use worst case assumptions about any call sites. Thus, the presence of any call in a routine eliminated potential constants along paths

⁴Because the multiple entry points in **ocean** were never called, they were eliminated in order to include **ocean** in the study.

⁵In each case, only one pass of dead code elimination was needed. The second propagation did not expose any newly dead code.

<i>Program</i>	<i>Using Return Jump Functions</i>				<i>No Return Jump Functions</i>	
	Polynomial	Pass-through	Intraprocedural	Literal	Polynomial	Pass-through
adm	110	110	110	110	110	110
doduc	289	289	289	288	287	287
fpppp	60	60	54	49	56	56
linpackd	170	170	170	94	170	170
matrix300	138	138	122	71	138	138
mdg	41	41	40	31	40	40
ocean	194	194	194	57	62	62
qcd	180	180	180	180	180	180
simple	183	183	179	174	183	183
snasa7	336	336	336	254	336	336
spec77	137	137	137	104	137	137
trfd	16	16	16	16	16	16

Table 2: Constants found through use of jump functions.

<i>Program</i>	<i>Polynomial without MOD</i>	<i>Polynomial with MOD</i>	<i>Complete Propagation</i>	<i>Intraprocedural Propagation</i>
adm	25	110	110	105
doduc	288	289	289	3
fpppp	34	60	60	38
linpackd	33	170	170	74
matrix300	18	138	138	69
mdg	31	41	41	31
ocean	79	194	204	56
qcd	169	180	180	179
simple	2	183	183	174
snasa7	303	336	336	254
spec77	76	137	141	83
trfd	10	16	16	15

Table 3: Comparison of most precise jump function with other propagation techniques.

leaving the call site, even when no modification occurred. The data indicates that incorporating MOD information significantly increases the number of constants that can be detected in scientific FORTRAN codes.

Incorporating interprocedural constants into a routine can expose segments of code that can never be executed. Removing this dead code can potentially eliminate conflicting definitions of variables and expose additional constants. Combining dead code elimination with interprocedural constant propagation in the “complete” propagation (see column 3) exposed few additional constants. The additional effort of a “complete” propagation does not appear justified by the additional constants found.

In fact, the results that we obtained in this study with “complete” propagation can be achieved by basing the jump-function generator on a *gated single-assignment form* [2, 10]. An analyzer based on gated single-assignment form would never consider the dead assignments that we found in the “complete” propagation.⁶ This would let the standard polynomial jump function produce the results seen with “complete” propagation.

Finally, the results of an intraprocedural constant propagation are presented to show the number of constants that it found. For fair comparison, MOD information was used in the intraprocedural propagation. For programs that contained constants, the interprocedural propagation (see column 2) always detected more constants than strictly intraprocedural propagation (see column 4). In some cases, the difference was substantial. In scientific FORTRAN codes, interprocedural constant propagation exposed many constants not found by the intraprocedural propagation.

5 Other Work

Several researchers have proposed techniques for propagating interprocedural constants. Wegman and Zadeck propose combining procedure integration with intraprocedural constant propagation to detect interprocedural constants [16]. Because procedure integration makes paths through the program’s call graph explicit, the interprocedural information computed along a particular path may be improved. By combining the results of the improved analysis with dead code elimination, it may be possible to eliminate some paths through the program. Data is not yet available to indicate whether or not the proposed algorithm

⁶Note that information from return jump functions is used during the construction of the gated single-assignment graph. Hence, a limited number of interprocedural constants may be available for this construction.

would perform efficiently in practice.

Callahan, Cooper, Kennedy, and Torczon present an interprocedural constant propagation framework that consists of an algorithm for propagating constants across call boundaries and a family of methods, called *jump functions*, used to propagate the constants through procedure bodies [4]. Both interprocedural constants passed to called procedures and those returned by a called procedure can be detected using this technique. Because this technique does not make paths through the call graph explicit, it potentially detects fewer constants than the method proposed by Wegman and Zadeck. This technique has been implemented in several compilation systems and has proved to be efficient in practice.

Burke and Cytron propose a similar approach for computing interprocedural constants within a unified framework that integrates dependence analysis with interprocedural analysis [3]. Intraprocedural constant propagation is employed to determine which constants are available for propagation at call sites. Interprocedural constants are propagated from call sites to the called procedure and used as initial information in the intraprocedural constant propagation. The method described does not handle returned constants or make paths through the call graph explicit.

Metzger and Stroud implemented interprocedural constant propagation in the CONVEX Application Compiler based on ideas presented in Callahan *et al.* [13]. They used interprocedural constants to guide procedure cloning. Their empirical results indicate that goal-directed cloning of procedures based on interprocedural constants can substantially increase the number of interprocedural constants available for use by later analysis and optimization passes. Experience with the CONVEX Application Compiler has shown that interprocedural constants can be used to increase the effectiveness of intraprocedural constant propagation, dependence analysis, dead code elimination, loop strip mining, loop interchange, and interprocedural optimization.

6 Conclusions

We have examined the effectiveness of four forward jump functions and one return jump function. The pass-through and polynomial parameter forward jump functions were equivalent in the number of constants that they found; in practice, the former technique should prove less expensive to build and evaluate. Incorporating MOD information is important; it led to the discovery of many additional constants. Incorporating dead code elimination into the process did not expose many additional constants.

Scientific FORTRAN programs use constants that cannot be detected with intraprocedural constant propagation. Our study showed that interprocedural constant propagation can discover such constants, even with simple jump functions. The difference between jump function implementations is significant.

Acknowledgements

We would like to thank the members of the ParaScope programming environment group at Rice, particularly Paul Havlak and Mary Hall, for providing the infrastructure used in this study. We would also like to thank Keith Cooper and Preston Briggs for their constant encouragement and support.

References

- [1] Bowen Alpern, Mark N. Wegman, and F. Kenneth Zadeck. Detecting equality of variables in programs. In *Conference Record of the Fifteenth Annual ACM Symposium on Principles of Programming Languages*, pages 1–11, January 1988.
- [2] Robert A. Ballance, Arthur B. Maccabe, and Karl J. Ottenstein. The program dependence web: A representation supporting control, data, and demand-driven interpretation of imperative languages. *SIGPLAN Notices*, 25(6):257–271, June 1990. *Proceedings of the ACM SIGPLAN '90 Conference on Programming Language Design and Implementation*.
- [3] Michael Burke and Ron Cytron. Interprocedural dependence analysis and parallelization. *SIGPLAN Notices*, 21(7):162–175, July 1986. *Proceedings of the ACM SIGPLAN '86 Symposium on Compiler Construction*.
- [4] David Callahan, Keith D. Cooper, Ken Kennedy, and Linda Torczon. Interprocedural constant propagation. *SIGPLAN Notices*, 21(7):152–161, July 1986. *Proceedings of the ACM SIGPLAN '86 Symposium on Compiler Construction*.
- [5] Keith D. Cooper, Mary W. Hall, Robert T. Hood, Ken Kennedy, Kathryn McKinley, John Mellor-Crummey, Linda Torczon, and Scott K. Warren. The ParaScope parallel programming environment. *Proceedings of the IEEE*, February 1993. (to appear).
- [6] Keith D. Cooper, Mary W. Hall, and Ken Kennedy. Procedure cloning. In *Proceedings of the IEEE Computer Society 1992 International Conference on Computer Languages*, pages 96–105, April 1992.
- [7] Keith D. Cooper and Ken Kennedy. Interprocedural side-effect analysis in linear time. *SIGPLAN Notices*, 23(7):57–66, July 1988. *Proceedings of the ACM SIGPLAN '88 Conference on Programming Language Design and Implementation*.
- [8] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on Programming Languages and Systems*, 13(4):451–490, October 1991.
- [9] Rudolf Eigenmann and William Blume. An effectiveness study of parallelizing compiler techniques. In *Proceedings of the 1991 International Conference on Parallel Processing*, St. Charles, IL, August 1991.
- [10] Paul Havlak. *Interprocedural Symbolic Analysis*. PhD thesis, Rice University, expected May, 1993.
- [11] Matthew S. Hecht. *Flow Analysis of Computer Programs*. Programming Languages Series. Elsevier North-Holland, Inc., 52 Vanderbilt Avenue, New York, NY 10017, 1977.
- [12] Gary A. Kildall. A unified approach to global program optimization. In *Conference Record of the ACM Symposium on Principles of Programming Languages*, pages 194–206, October 1973.
- [13] Robert Metzger and Sean Stroud. Interprocedural constant propagation: An empirical study. (To appear in *ACM Letters on Programming Languages and Systems*).
- [14] Zhiyu Shen, Zhiyuan Li, and Pen-Chung Yew. An empirical study of FORTRAN programs for parallelizing compilers. *IEEE Transactions on Parallel and Distributed Systems*, 1(3):356–364, July 1990.
- [15] Jaswinder P. Singh and John Hennessy. An empirical investigation of the effectiveness of and limitations of automatic parallelization. In *Proceedings of the International Symposium on Shared Memory Multiprocessors*, Tokyo, Japan, April 1991.
- [16] Mark N. Wegman and F. Kenneth Zadeck. Constant propagation with conditional branches. *ACM Transactions on Programming Languages and Systems*, 13(2):181–210, April 1991.